

Перевірка синхронізації потоків у лабораторній роботі №15

Об'єктно-орієнтоване програмування. Java

Синхронізація потоків у Java

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Володимир Миколайович Савченко

Рік: 2026

1. Мета перевірки

Метою цієї демонстрації є перевірка роботи синхронізації потоків у лабораторній роботі №15.

У моєму варіанті реалізовано лічильник подій. Кілька потоків одночасно збільшують один спільний лічильник. Без синхронізації частина оновлень може втрачатися, а з використанням `synchronized` фінальний результат стає правильним.

2. Демонстраційний файл

Для наочної перевірки синхронізації можна створити окремий демонстраційний файл:

`CounterSynchronizationDemo.java`

У цьому файлі показано два сценарії:

1. **Unsafe update without synchronized** — оновлення спільного лічильника без синхронізації.
2. **Safe update with synchronized** — оновлення спільного лічильника з використанням `synchronized`.

Демонстраційний файл потрібен для того, щоб на малому прикладі показати саму суть проблеми: два потоки можуть одночасно прочитати однакове значення спільної змінної і записати однаковий результат.

3. Unsafe-варіант без `synchronized`

У першій частині демонстрації два потоки `UnsafeCounter-1` та `UnsafeCounter-2` одночасно працюють зі спільною змінною `unsafeCounter`.

Обидва потоки читають початкове значення `unsafeCounter = 0`. Після цього кожен потік окремо обчислює нове значення 1 і записує його назад у спільну змінну.

Через це один інкремент втрачається: очікувалося значення 2, але фактично отримано 1.

Такий результат демонструє стан перегонів. Операція збільшення лічильника не є атомарною, бо складається з трьох окремих дій:

1. прочитати поточне значення;

2. обчислити нове значення;
3. записати результат назад.

Якщо між цими діями перемикаються потоки, один потік може перезаписати результат іншого.

```
=== 1. Unsafe update without synchronized ===
UnsafeCounter-1 read unsafeCounter = 0
UnsafeCounter-2 read unsafeCounter = 0
UnsafeCounter-2 calculated updated value = 1
UnsafeCounter-1 calculated updated value = 1
UnsafeCounter-2 wrote unsafeCounter = 1
UnsafeCounter-1 wrote unsafeCounter = 1
Expected unsafeCounter = 2
Actual unsafeCounter = 1
Race condition detected = true
```

Рисунок 1 — Демонстрація race condition без synchronized. Два потоки одночасно прочитали однакове значення, тому один інкремент було втрачено.

4. Safe-варіант із synchronized

У другій частині демонстрації використовується synchronized-блок.

Потік SafeCounter-1 першим заходить у критичну секцію та працює зі спільною змінною safeCounter. У цей час потік SafeCounter-2 намагається зайти в той самий synchronized-блок, але не може цього зробити, тому що монітор уже зайнятий першим потоком.

Через це другий потік переходить у стан BLOCKED і чекає, доки перший потік звільнить монітор.

У цьому випадку результат є правильним: очікувалося 2 і фактично отримано 2.

Це означає, що synchronized не дозволив двом потокам одночасно виконувати критичну секцію.

```
=== 2. Safe update with synchronized ===
SafeCounter-1 tries to enter synchronized block
SafeCounter-1 entered synchronized block
SafeCounter-1 read safeCounter = 0
SafeCounter-2 tries to enter synchronized block

--- Thread states while synchronized block is locked ---
SafeCounter-1 state = TIMED_WAITING
SafeCounter-2 state = BLOCKED
If SafeCounter-2 is BLOCKED, it is waiting for the synchronized lock.

SafeCounter-1 wrote safeCounter = 1
SafeCounter-1 leaves synchronized block
SafeCounter-2 entered synchronized block
SafeCounter-2 read safeCounter = 1
SafeCounter-2 wrote safeCounter = 2
SafeCounter-2 leaves synchronized block
Expected safeCounter = 2
Actual safeCounter = 2
Synchronization works = true
```

Рисунок 2 — Демонстрація роботи synchronized. Потік SafeCounter-1 увійшов у критичну секцію, а потік SafeCounter-2 має стан BLOCKED і очікує звільнення монітора.

5. Пояснення станів потоків

Під час перевірки були використані стани потоків TIMED_WAITING і BLOCKED.

Стан TIMED_WAITING означає, що потік тимчасово очікує. У цій демонстрації такий стан виникає через виклик Thread.sleep().

Стан BLOCKED означає, що потік не може увійти в synchronized-блок, тому що монітор уже зайнятий іншим потоком.

У safe-варіанті це нормальна поведінка: один потік виконує критичну секцію, а другий очікує доступу до неї.

6. Зв'язок із основною програмою

Основна програма лабораторної роботи працює з класами SharedCounter, UnsafeSharedCounter, SafeSharedCounter, EventSource і CounterTask.

У класі UnsafeSharedCounter метод increment() не синхронізований, тому кілька потоків можуть одночасно змінювати поле value. Це може призвести до втрати оновлень.

У класі SafeSharedCounter методи increment(), getValue() і reset() позначені ключовим словом synchronized. Завдяки цьому тільки один потік одночасно може змінювати значення лічильника.

У демонстраційній програмі Main запускаються 4 потоки, кожен із яких виконує 100000 збільшень спільного лічильника. Очікуваний результат:

$$4 * 100000 = 400000$$

У небезпечному варіанті фактичний результат може бути меншим за очікуваний. У синхронізованому варіанті фактичний результат має дорівнювати 400000.

7. Висновок

У демонстрації було показано різницю між небезпечним і синхронізованим доступом до спільного ресурсу.

У unsafe-варіанті два потоки одночасно прочитали однакове значення unsafeCounter, тому фактичний результат став 1 замість очікуваного 2. Це підтверджує виникнення race condition.

У safe-варіанті використання synchronized призвело до того, що один потік виконував критичну секцію, а другий очікував у стані BLOCKED.

У результаті значення safeCounter стало правильним: очікувалось 2, фактично отримано 2.

Отже, synchronized захищає критичну секцію і не дозволяє кільком потокам одночасно змінювати спільний змінюваний стан.